

SME Library MCP — Implementation Checklist

What it takes to stand up and run the Subject Matter Expert MCP service · Vercel + Supabase

This is the deployment and operations runbook for the SME Library MCP server. Items marked code done are implemented in the repository; items marked action are steps *you* perform to get a live service. Work top to bottom.

Architecture in one line: a Next.js app on Vercel serves MCP over Streamable HTTP at `/api/mcp`, backed by a standalone Supabase project (Postgres + Auth + pgvector + Edge Functions). Agents authenticate with hashed bearer API keys; humans use a Supabase-Auth dashboard to mint keys.

1 · Prerequisites

- ☐ A **Supabase** account and a new, dedicated project (standalone — not shared with other apps)
- ☐ A **Vercel** account connected to your git repository
- ☐ An **Anthropic API key** (used by the `generate_sme` tool)
- ☐ The **Supabase CLI** installed locally (`npm i -g supabase`) and `supabase login` done
- ☐ Node.js 18+ locally for the bootstrap script and optional local runs

2 · Supabase — database

- ☐ Create the project; note the project ref, URL, `anon` key, and `service_role` key
- ☐ `supabase link --project-ref <ref>`
- ☐ Apply all migrations: `supabase db push`code done (migrations 001–003 in repo)
- ☐ Confirm the `vector` and `pgcrypto` extensions enabled (migration 001 does this)
- ☐ Confirm the seeded library workspace row exists (id `0000...0001`)
- ☐ Verify RLS is enabled deny-all on every table (migration 001) and the authenticated read policies exist (migration 003)

Schema shipped in the repo code done

Table	Purpose
<code>workspaces</code>	Tenants; one seeded library workspace
<code>api_keys</code>	SHA-256-hashed keys, scopes, tier, expiry, revocation
<code>smes</code>	Domain-agnostic profiles + <code>extensions</code> jsonb, FTS vector, embedding
<code>sme_versions</code>	Full snapshot per edit (rollback/diff)
<code>sme_feedback</code>	Raw per-session scores → Bayesian quality
<code>library_promotions</code>	Moderated promotion queue
<code>workspace_members</code>	Auth users → workspaces
<code>audit_log</code> , <code>rate_limits</code>	Audit trail; fixed-window rate limiting

3 · Supabase — Auth & Edge Functions

- ☐ Enable **Email** auth provider (Authentication → Providers). Decide whether to require email confirmation
- ☐ Add your Vercel domain to Auth → URL Configuration → Redirect URLs
(`https://<domain>/auth/callback`)
- ☐ Deploy the embedding function: `supabase functions deploy embed``code done` (gte-small, 384-dim) — optional; search falls back to keyword FTS if absent

4 · Vercel — environment variables

Variable	Required	Purpose
<code>SUPABASE_URL</code>	yes	Project URL
<code>SUPABASE_SERVICE_ROLE_KEY</code>	yes	Server-side DB access (never exposed to client)
<code>SUPABASE_ANON_KEY</code>	yes	Powers the signed-in dashboard (RLS applies)
<code>ANTHROPIC_API_KEY</code>	yes	<code>generate_sme</code> tool
<code>CRON_SECRET</code>	yes	Protects <code>/api/cron/maintenance</code>
<code>ADMIN_EMAILS</code>	rec.	Comma-separated admins for the promotion queue
<code>ANTHROPIC_MODEL</code>	no	Override; defaults to <code>claude-opus-4-8</code>
<code>NEXT_PUBLIC_BASE_URL</code>	no	Shown in connect snippets

- ☐ Set the above in the Vercel project (Settings → Environment Variables)
- ☐ Generate a long random `CRON_SECRET` (e.g. `openssl rand -hex 32`)

5 · Deploy

- ☐ Import the repo into Vercel (framework auto-detected as Next.js)
- ☐ `vercel deploy --prod` (or push to the production branch)
- ☐ Confirm the daily cron is registered from `vercel.json``code done` (Settings → Cron Jobs)
- ☐ Hit `GET /api/health` — expect `configured.supabase` and `configured.anthropic` both `true`

6 · First keys & smoke test

- ☐ Mint an operator/admin key: `node scripts/create-key.mjs --workspace "Ops" --admin``code done`
- ☐ Or self-serve: visit `/dashboard` , sign up, create a read/write key
- ☐ Connect a client:
`claude mcp add --transport http sme-library https://<domain>/api/mcp --header "Authorization: Bearer sme_live_..."`
- ☐ Exercise each tool: `create_sme` → `search_smes` → `get_sme` → `update_sme` → `generate_sme` → `record_feedback` → `propose_promotion` → `clone_sme` → `archive_sme`
- ☐ As an admin, approve a proposal at `/dashboard/admin` ; confirm it appears in `GET /api/library`
- ☐ Confirm auth is enforced: unauthenticated `/api/mcp` and `/api/cron/maintenance` return 401

7 · What's already built code done

Phase	Capability
1	HTTP transport (mcp-handler), hashed-key auth, rate limits, 11 tools, stdio for local dev
2	Supabase Auth dashboard: signup, auto-workspace, key mint/revoke
3	Hybrid search: keyword FTS + pgvector semantic + reciprocal rank fusion
4	Daily cron: quality recompute, rate-limit cleanup, embedding backfill
5	Admin promotion-review UI; cached public <code>/api/library</code> endpoint

The 11 MCP tools code done

`search_smes` · `list_smes` · `get_sme` · `create_sme` · `update_sme` · `archive_sme` · `clone_sme` · `generate_sme` · `record_feedback` · `propose_promotion` · `review_promotion` (admin)

8 · Remaining engineering work action

- ☐ **OAuth 2.1** (dynamic client registration + PKCE) so claude.ai custom connectors authorize interactively instead of pasting a key — `mcp-handler` supports it via `withMcpAuth`; needs Supabase configured as an OAuth authorization server
- ☐ **Stale-SME decay**: flag SMEs unused for N months / declining scores for deprecation review (extend the cron)
- ☐ **LLM content screen on promotion**: auto-reject personas designed for harm before they enter the library
- ☐ **Usage dashboards**: per-workspace metering views over `audit_log`
- ☐ **report_sme** path for community moderation of library entries
- ☐ **Configurable rate tiers**: per-key `rate_limit_tier` (free/pro/unlimited) for self-hosters to tune quotas — no billing, open source
- ☐ **Automated tests + CI**: unit tests for auth/search/promotions; CI on PR

9 · Operations

- ☐ Verify the maintenance cron runs (Vercel → Cron logs); it recomputes quality, prunes rate-limit windows, backfills embeddings
- ☐ Rotate leaked keys via the dashboard (revoke) or DB; keys are hashed so plaintext can't be recovered
- ☐ Monitor `audit_log` for anomalous write volume per key
- ☐ Back up the Supabase database (point-in-time recovery on paid plans)
- ☐ Watch Anthropic spend — `generate_sme` is quota-limited per key but bills to your account